Profitable Growth Under Congestion Constraints

A Personal SQL Strategy Case Study for NYC Ride Hailing

Master SQL Analysis Documentation

Written by Sai Sourabh Akula
github.com/sourabhakula/nyc-rideshare-strategy

Database: nyc_rideshare | Period: December 1 to 31, 2025 | Approximately 100,000 Trips
Platforms Analyzed: Uber (HV0003) and Lyft (HV0005)

This document is a full record of how I built this project from scratch. It covers every data definition, every assumption I made and why, every calculation with its formula and business rationale, and every query explained in plain language. I wrote this so that anyone reading it, including a future version of myself, can understand exactly what I did, why I did it, and what conclusions the data supports.

Table of Contents

Section 1 Where This Data Comes From and What It Means

Section 2 The Problem I Set Out to Solve

Section 3 Every Assumption I Made and the Reason Behind Each One

Section 4 Every Calculation I Used and Why I Calculated It That Way

Section 5 The Numbers I Verified and Use as My Ground Truth

Section 6 The Six Most Important Things I Found

Query 01 Market Position Scorecard

Query 02 Demand Heatmap by Borough and Time of Day

Query 03 Origin to Destination Corridor Analysis

Query 04 Take Rate by Fare Decile

Query 05 Passenger Price Decomposition

Query 06 Driver Pay Stress Test

Query 07 Congestion Burden Analysis

Query 08 Holiday Demand Stress and Revenue Volatility

Query 09 Price Volatility Analysis

Query 10 Zone Opportunity Matrix

Query 11 Growth Quality Score

Appendix Output Section Labels and APA References

SECTION 1

Where This Data Comes From and What It Means

Before I wrote a single line of SQL for this project, I spent time understanding exactly what I was working with and what the data was actually capable of telling me. I find that rushing into queries without grounding yourself in the source material is one of the most common mistakes analysts make, and it tends to produce confident-looking numbers that fall apart the moment someone asks a follow-up question. So I want to document everything I learned about this dataset before explaining any of the analysis.

The data comes from the New York City Taxi and Limousine Commission, which everyone in this space calls the TLC. The TLC is the city agency that regulates every for-hire vehicle operating in the five boroughs, from yellow cabs to app-based platforms like Uber and Lyft. Under New York City Administrative Code Section 19-548, every High Volume For-Hire Vehicle base is legally required to submit detailed trip records to the TLC on an ongoing basis. A High Volume For-Hire Vehicle, or HVFHV, is defined as any platform that completes ten thousand or more trips per day. This legal requirement is what makes this dataset possible and what makes it so rich. These are not sampled records. This is every single trip that Uber and Lyft completed in New York City during the period covered.

The specific dataset I am working with covers December 2025. I chose December deliberately. It is analytically the most interesting single month in the ride hailing calendar. Within those 31 days you get a pre-holiday demand surge in the second week, a sharp Christmas trough around December 25th, a recovery pattern in the last week of the month, and New Year's Eve at the very end. On top of that, the NYC Central Business District congestion pricing program became effective in January 2025, which means December data captures the early steady-state behavior of platforms operating under that new fee regime. No other single month gives you this much analytical texture in one place.

The dataset has been loaded into a MySQL database called nyc_rideshare. There are two tables that all my queries work with. The primary table is called trips and contains approximately 100,000 rows after filtering to Uber and Lyft only. The second table is taxi_zone_lookup, which is a reference table that maps each TLC zone ID to a borough name, neighborhood name, and service zone type. This lookup table is essential because the trips table stores locations as numeric zone IDs, not human-readable names. I join it twice in my view, once for the pickup location and once for the dropoff location.

I want to be explicit about one thing regarding the platform identification. The raw data does not store the name Uber or Lyft anywhere in the trips table. Instead, each platform is represented by its TLC High Volume For-Hire Vehicle base license number. Uber operates under license HV0003 and Lyft operates under HV0005. This mapping is confirmed by the TLC's publicly maintained HVFHV base license registry (NYC TLC, 2024). Throughout my analysis, I decode these license numbers to brand names using a CASE statement so that every output is readable to a business audience.

MySQL version 8.0 or higher is required to run all of my queries. This is not a minor technical detail. Several of my most important queries use window functions such as RANK, LAG, NTILE, and rolling

AVG with ROWS BETWEEN clauses. These features were introduced in MySQL 8.0 and do not exist in earlier versions. If you try to run this on MySQL 5.7 or earlier, the window function queries will fail entirely.

Complete Data Dictionary for the Trips Table

I am going to define every column in the trips table that is relevant to my analysis, because this is the most important reference section in this entire document. When a number looks surprising, the explanation is almost always in one of these column definitions.

`hvfhs_license_num`
The TLC HVFHV base license number. This is my primary platform identifier. HV0003 maps to Uber and HV0005 maps to Lyft. All other values are excluded by the view filter.

`dispatching_base_num`
The specific TLC base that dispatched this trip. One platform can operate multiple bases, so Uber alone has several base numbers. I ignore this column and work at the platform level using hvfhs_license_num.

`originating_base_num`
The base that originally received the ride request. This column is NULL on every single row in this dataset. I confirmed this and excluded it from all analysis. Any query referencing it returns NULL for all rows.

`request_datetime`
The exact timestamp when the rider placed the trip request through the app. This is the start of the customer experience. I use it to compute how long the rider waited before being picked up.

`on_scene_datetime`
When the driver physically arrived at the pickup location. I chose not to use this for my SLA metric because it represents when the driver reached the general block, not when the rider actually got in the car. The request to pickup gap is a more complete measure of the rider experience.

`pickup_datetime`
When the rider entered the vehicle and the trip officially started. This combined with request_datetime gives me the wait time that the rider actually experienced.

`dropoff_datetime`
When the rider exited the vehicle and the trip ended. I use this to classify trips by time of day based on the pickup hour. I do not use dropoff for my SLA calculation.

`PULocationID`
The TLC numeric zone ID for the pickup location. I join this to taxi_zone_lookup to get the borough and neighborhood name of where the trip started.

`DOLocationID`
The TLC numeric zone ID for the dropoff location. I join this to taxi_zone_lookup separately to get the destination borough and zone for corridor analysis.

`trip_miles`
The GPS-tracked trip distance in statute miles. I filter out rows where this is zero or null because a zero-mile trip is a data recording error, not a real trip. I use this for fare per mile calculations and for the TLC minimum pay

formula.

trip_time

The trip duration stored in SECONDS. This is the most critical data type detail in the entire dataset. The column name suggests minutes or time generically, but the values are in seconds. A value of 138 equals 2.3 minutes, confirmed by cross-referencing with actual trip timestamps. Every calculation that needs minutes divides this by 60. Every hourly rate calculation multiplies by 3600.

base_passenger_fare

The platform-controlled price charged to the rider for the trip itself. This excludes tolls, government fees, taxes, and tips. This is the single most important column in my analysis because it is what the platform actually controls and what all margin calculations are based on.

tolls

Road, bridge, and tunnel tolls paid during the trip. These are a pure pass-through to the rider. The platform earns zero from tolls. I treat NULL as zero using COALESCE.

bcf

The Black Car Fund fee. This is a 1.5 percent surcharge on the base fare mandated by the New York Workers Compensation Board to fund injury coverage for for-hire vehicle drivers (Black Car Fund, 2024). I treat NULL as zero.

sales_tax

New York State and New York City combined sales tax applied to the base fare. The combined rate is approximately 8.875 percent. Because it is calculated as a percentage of the base fare, it scales proportionally with trip value unlike the flat fees. I treat NULL as zero.

congestion_surcharge

A state-mandated flat surcharge applied to trips that begin, end, or pass through Manhattan south of 96th Street. This was established under New York State Tax Law Section 1299-B in 2019. The typical rate ranges from 0.75 to 2.75 dollars per trip depending on vehicle type. I treat NULL as zero.

airport_fee

A flat surcharge charged on trips to or from JFK Airport, LaGuardia Airport, or Newark Liberty International Airport. The typical value is approximately 2.50 dollars. I use airport_fee greater than zero as my proxy for identifying airport trips, validated by cross-checking with pickup zone names containing the word Airport.

cbd_congestion_fee

The new Central Business District congestion pricing fee that became effective January 5, 2025 under the MTA Central Business District Tolling Program. It applies to trips that enter Manhattan below 60th Street. This is a flat per-trip fee, typically around 1.49 dollars on HVFHV trips. I treat NULL as zero.

tips

The tip amount paid by the rider through the app. One hundred percent of this amount goes to the driver. The platform earns nothing from tips. I include tips in the total passenger charge calculation because it is part of what riders pay out of pocket, but I exclude it from any platform margin calculation. I treat NULL as zero.

driver_pay

The total payment made to the driver for completing this trip. This amount is set according to TLC minimum pay rules and includes both per-mile and per-minute components. It does not include tips. This column and base_passenger_fare together are the foundation of every platform economics calculation in my analysis.

shared_request_flag

A Y or N flag indicating whether the rider requested a shared or pooled ride. I found this data to have very low and unreliable volume in December 2025, so I excluded it from core analysis.

shared_match_flag

A Y or N flag indicating whether the shared ride request was successfully matched with another rider. Same limitation as shared_request_flag applies.

access_a_ride_flag

A Y or N flag for trips completed under NYC's Access-A-Ride paratransit program, which provides subsidized rides for disabled riders. Low volume in this dataset.

wav_request_flag

A Y or N flag indicating whether the rider requested a wheelchair accessible vehicle. This column is critical for my equity and accessibility analysis in Query 10. The absence of a request does not mean there is no need. Riders may have given up requesting WAV vehicles because they learned their requests were never fulfilled.

wav_match_flag:

A Y or N flag indicating whether the WAV request was successfully matched to an actual wheelchair accessible vehicle. The gap between wav_request_flag equal to Y and wav_match_flag equal to Y is the unmet accessibility need I quantify in Query 10.

The Taxi Zone Lookup Table

The taxi_zone_lookup table is a static reference table published by the TLC that maps each numeric zone ID to a readable location. It has four columns. LocationID is the integer that matches PULocationID and DOLocationID in the trips table. Borough is the NYC borough name, which can be Manhattan, Brooklyn, Queens, Bronx, or Staten Island, plus EWR for Newark Airport trips and N/A for out-of-city zones. Zone is the neighborhood or landmark name such as JFK Airport or Midtown Center or Kingsbridge Heights. service_zone categorizes the zone type as Boro Zone, Yellow Zone, Airports, or EWR. I join this table twice in my view using LEFT JOIN so that every trip retains its pickup and dropoff zone information even in cases where the zone ID does not match a known entry in the lookup table.

One important note on borough scope: when my queries group by pu_borough, I always include a WHERE pu_borough IS NOT NULL clause because a small number of trips have PULocationIDs that do not match any row in the lookup table. These produce NULL boroughs and would otherwise create a confusing catch-all group in aggregated results.

SECTION 2

The Problem I Set Out to Solve

I built this project because I wanted to practice the kind of multi-stakeholder analytical thinking that comes up in real business environments, where the same data needs to answer different questions for different audiences simultaneously. I could have just run some aggregate queries and called it a portfolio project. Instead, I chose to frame it around a genuine strategic tension that exists in the NYC ride hailing market in December 2025 and to build all eleven of my analysis queries around that tension.

The core tension is this: NYC's ride hailing market is under pressure from three directions at the same time. First, both major platforms are experiencing structural margin erosion, meaning they are losing money on a significant percentage of every trip they complete. In Uber's case, I found that 18.23 percent of all trips had a negative take rate, meaning the driver was paid more than the rider paid the platform. That is a real problem that volume growth alone cannot hide forever. Second, the NYC TLC has regulatory responsibilities around driver pay fairness and rider equity that create friction with platform expansion goals. The TLC minimum pay standard, the new congestion pricing fee, and the WAV accessibility requirement all represent constraints that platforms must operate within. Third, the market structure leaves genuine gaps that a well-capitalized new entrant could exploit if it understood which zones and time windows are underserved by the incumbents.

These three pressures do not exist in isolation. They interact with each other in ways that make simple analysis insufficient. A zone that looks like a great entry opportunity on volume alone might turn out to be one where no platform makes money because driver pay requirements eat all the margin. A borough that appears to carry high congestion fee burden on the surface might reveal, on closer inspection, that the burden is actually distributed by trip length rather than geography. Getting these nuances right requires looking at the data from multiple angles simultaneously, which is exactly what I designed the eleven-query structure to do.

The Four Stakeholder Perspectives

I organized all eleven queries around four distinct stakeholder perspectives. Each perspective has a different core question and a different set of metrics that matter most to them. This is not just a framing device. It actually changed which calculations I built, which thresholds I set, and which findings I chose to surface.

The first perspective is the Uber CEO. The central question from this lens is: we dominate volume, but are we actually maximizing value per trip? Where are we leaking margin, and is our December growth built on a solid foundation or a fragile one? This perspective drives Queries 01, 04, 08, and 11.

The second perspective is the Lyft CEO. Lyft is the smaller platform by a significant margin, with about 27 percent of market share against Uber's 73 percent. The question here is: where is our competitive edge genuinely defensible? Which boroughs, corridors, and time windows should we double down on rather than spreading resources across the whole market? This perspective drives Queries 02, 03, 05, and 09.

The third perspective is the NYC TLC Commissioner. The regulatory lens asks: are platforms paying drivers at or above the minimum standard I have set? Are my congestion pricing policies creating inequitable burdens on riders? Which operators and segments need enforcement attention? This perspective drives Queries 06 and 07.

The fourth perspective is a hypothetical new market entrant, a well-funded company considering launching ride hailing services in NYC. The question here is: where is demand structurally unmet by both incumbents? Which zones have high fares, long waits, and no platform serving them well? What does a first-year entry strategy look like based on the data? This perspective drives Queries 09, 10, and 11.

SECTION 3

Every Assumption I Made and the Reason Behind Each One

This section is the one I would want to read if I were picking this project apart or inheriting it from someone else. Every analytical decision involves assumptions, and the assumptions that go undocumented are the ones that cause confusion later. I have documented all sixteen of the assumptions I made in this project, including the reference or reasoning that justifies each one.

Assumption 1: Platform Identification via License Number

I treat HV0003 as Uber and HV0005 as Lyft throughout the entire analysis. This mapping is confirmed by the TLC's publicly maintained HVFHV base license registry, which lists the operating name associated with each license number (NYC TLC, 2024). Any trip with a license number other than HV0003 or HV0005 is excluded entirely by the WHERE clause in my view. This keeps the analysis clean and focused on the two dominant consumer platforms. There are other HVFHV operators in NYC, but none with the volume or market presence to be strategically relevant to the questions I am answering.

Assumption 2: trip_time Is Stored in Seconds

This is the most critical technical assumption in the project and the one most likely to cause errors if missed. The column is named trip_time and contains integer values, but those values represent seconds, not minutes or any other unit. I confirmed this by cross-referencing sample rows: a trip_time value of 138 combined with a pickup_datetime and dropoff_datetime difference of approximately 2.3 minutes confirms the seconds interpretation. The implication is that every calculation involving duration must divide by 60.0 to get minutes, and every hourly rate calculation must multiply by 3600. I perform this conversion once in the view and store it as trip_min, so all downstream queries use the preconverted value.

> *Murphy, T. (2019). Data type conventions in relational databases. ACM SIGMOD Record, 48(2), 12-19.*

Assumption 3: I Measure Wait Time as Request to Pickup, Not On-Scene to Pickup

The TLC data includes both on_scene_datetime, which is when the driver arrived at the pickup location, and request_datetime, which is when the rider made the booking. I chose to measure wait time as the gap from request to pickup rather than on-scene to pickup. The reason is that the request to pickup window is the total time the rider experiences as waiting. From the rider's perspective, the wait begins the moment they request the trip. The on-scene time is relevant to understanding driver behavior but it does not capture what the rider goes through. For service quality analysis and SLA measurement, the rider experience is the correct metric (Parasuraman, Zeithaml, and Berry, 1988).

> *Parasuraman, A., Zeithaml, V. A., & Berry, L. L. (1988). SERVQUAL: A multiple-item scale for measuring consumer perceptions of service quality. Journal of Retailing, 64(1), 12-40.*

Assumption 4: My SLA Standard Is Five Minutes from Request to Pickup

I define a successful SLA outcome as the rider receiving a pickup within five minutes of placing their request. The binary flag sla_5min_met equals 1 when the wait is 300 seconds or less and 0 otherwise. This five-minute threshold is the industry standard for urban on-demand ride hailing service quality.

Research by Hall, Horton, and Knoepfle (2017) on Uber's platform found that five-minute pickups are the benchmark riders use to evaluate reliability. The NYC TLC itself has referenced this threshold in its rider service quality assessments (NYC TLC, 2023). Any SLA percentage I report is simply AVG(sla_5min_met) multiplied by 100 for a given segment.

Hall, J. V., Horton, J. J., & Knoepfle, D. T. (2017). Pricing in designed markets: The case for ride-sharing. Working Paper, Harvard University. NYC TLC. (2023). 2023 factbook. New York City Taxi and Limousine Commission.

Assumption 5: I Identify Airport Trips Using airport_fee Greater Than Zero

The trips table does not have a dedicated boolean airport trip flag. Instead, I identify airport trips by checking whether the airport_fee column has a positive value. The TLC mandates that this fee, typically around 2.50 dollars, is only charged on qualifying trips to or from JFK, LaGuardia, or Newark Liberty International Airport. Therefore, any trip with airport_fee greater than zero must be an airport trip by definition. I validated this assumption by cross-referencing trips where is_airport_trip equals 1 with their PULocationID and DOLocationID values, and confirmed they matched zones named JFK Airport, LaGuardia Airport, or Newark Airport in the lookup table.

NYC TLC. (2024). High volume for-hire vehicle rules: Chapter 59B of the rules of the city of New York.

Assumption 6: I Use Aggregate SUM-Based Take Rate, Never AVG of Per-Trip Rates

This is the single most important methodological assumption in my analysis, and I want to explain it in full because it changes every platform-level and segment-level take rate number significantly. Take rate is the percentage of the base passenger fare that the platform keeps after paying the driver. There are two ways to calculate this at the aggregate level: you can average the per-trip rates, or you can compute it on the total sums. The average method is wrong for this use case. Here is why. Imagine two trips: a 4 dollar trip where the driver earns 4.50 dollars, giving a take rate of negative 12.5 percent, and a 120 dollar trip where the driver earns 85 dollars, giving a take rate of positive 29.2 percent. The simple average of these two rates is 8.35 percent. But the platform actually collected 124 dollars in base fare and paid out 89.50 dollars in driver pay, keeping 34.50 dollars, which is a true take rate of 27.8 percent. The average method understates the real margin by more than 19 percentage points. The correct method is always to compute SUM of base fare minus SUM of driver pay divided by SUM of base fare at the segment level. I call this the aggregate method. I validated it by comparing results: Uber's Bronx take rate using the aggregate method came out to 11.01 percent against a ground truth of 10.99 percent, a difference of two basis points. The average method produced 14.21 percent for the same segment, an error of over 300 basis points. This matters enormously for business decisions.

Varian, H. R. (2014). Big data: New tricks for econometrics. Journal of Economic Perspectives, 28(2), 3-28.

Assumption 7: The TLC Minimum Pay Formula Is Miles Times 0.82 Plus Minutes Times 0.57

The NYC TLC established a minimum earnings standard for app-based drivers in 2018, and it has been updated periodically since then. The formula calculates a minimum amount the driver must receive per trip based on the trip's distance and time. For December 2025, the applicable rates are 0.82 dollars per mile and 0.57 dollars per minute (NYC TLC, 2024). A trip is flagged as a violation when driver_pay is less than the result of this formula. I compute this minimum in the view and store the violation status as a binary flag called below_tlc_minimum. A value of 1 means the driver was paid below the legal

standard on that trip.

NYC TLC. (2024). Minimum payment standard for high volume for-hire vehicle drivers. NYC TLC. Mishel, L. (2018). Uber and the labor market. Economic Policy Institute.

Assumption 8: I Track Two Separate Congestion Fees Because They Are Legally Distinct

There are two separate government-mandated congestion-related fees in this dataset and I treat them separately throughout my analysis before combining them for burden calculations. The congestion_surcharge column reflects a fee established in 2019 under New York State Tax Law Section 1299-B for trips in or through the Manhattan central business district (New York State, 2019). The cbd_congestion_fee column reflects the newer MTA Central Business District Tolling Program fee that became effective January fifth 2025 (MTA, 2025). These are legally different instruments with different legislative histories, different rate structures, and different geographic footprints, even though they both fall on riders in overlapping parts of Manhattan. For burden analysis I combine them as total_congestion_fees because their combined effect on the rider is what matters for equity analysis.

New York State. (2019). Tax Law Section 1299-B. MTA. (2025). Central business district tolling program: Frequently asked questions. Metropolitan Transportation Authority.

Assumption 9: My View Excludes Five Categories of Data Quality Problems

I built five explicit data quality filters into the WHERE clause of my view. First, I exclude any trip where pickup_datetime, dropoff_datetime, or request_datetime is NULL. Without these timestamps I cannot compute wait time, time of day, or trip duration. Second, I exclude trips where base_passenger_fare or driver_pay is NULL. These are the two core economic columns and every financial metric depends on them. Third, I exclude trips where trip_miles equals zero or trip_time equals zero. A zero-mile or zero-second trip is a recording error, not a real trip. Fourth, I exclude trips where the calculated wait time is either negative or greater than 7200 seconds. A negative wait time would mean the driver picked up the rider before the rider requested the trip, which is impossible and indicates a timestamp error. A wait time over two hours is almost certainly a data artifact rather than a real case. Fifth, I restrict to HV0003 and HV0005 only. These five filters together remove a small percentage of rows but ensure that every row I analyze represents a real, measurable trip.

Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. IEEE Data Engineering Bulletin, 23(4), 3-13.

Assumption 10: NULL in Optional Fee Columns Means Zero Dollars, Not Missing Data

Several of the fee columns in the trips table, specifically tolls, bcf, sales_tax, congestion_surcharge, airport_fee, cbd_congestion_fee, and tips, can appear as NULL. I treat all of these as zero using COALESCE in the view. The reason is that a NULL in these columns does not indicate that the data was not collected or is unreliable. It indicates that the fee was not applicable to that specific trip. For example, a trip originating in Staten Island with NULL congestion_surcharge genuinely charged zero in congestion surcharges because the trip did not originate in or pass through Manhattan. This is not missing data. Treating NULL as zero is the semantically correct interpretation for these columns.

Assumption 11: originating_base_num Is Completely Absent from This Dataset

The originating_base_num column, which is supposed to indicate which TLC base originally received the ride request before dispatching, is NULL on every single row in this dataset. I confirmed this by

running a COUNT of non-null values, which returned zero. I mention this explicitly because it is the kind of thing that can cause confusion if someone tries to use it without checking first. I excluded it from the view entirely.

Assumption 12: I Define Time of Day in Five Bins Based on Pickup Hour

I classify every trip into one of five time of day periods based on the hour of its pickup_datetime. AM Rush covers hours 6 through 9 inclusive, which aligns with the morning commute window. Midday covers hours 10 through 15, representing the off-peak daytime period. PM Rush covers hours 16 through 19, the evening commute. Evening covers hours 20 through 22, the post-dinner and social activity window. Late Night covers everything else, meaning hours 23 through 5, the overnight and bar-close period. These bins are aligned with the NYC MTA's peak and off-peak fare window definitions and with the standard time of day segmentation used in urban transportation research (Schaller, 2017). I classify by pickup hour, not request hour or dropoff hour, because the trip's demand and supply context are most relevant at the moment the ride begins.

Schaller, B. (2017). Empty seats, full streets: Fixing Manhattan's traffic problem. Schaller Consulting.

Assumption 13: I Use Coefficient of Variation as My Volatility Metric, Not Raw Standard Deviation

For price volatility analysis I use the coefficient of variation, which is the standard deviation divided by the mean multiplied by 100, rather than raw standard deviation. The reason is that fare levels differ substantially across zones, and comparing raw standard deviations across zones with different average prices is misleading. A zone with an average fare of 70 dollars and a standard deviation of 10 dollars has a CV of 14 percent, indicating relatively stable pricing. A zone with an average fare of 20 dollars and a standard deviation of 8 dollars has a CV of 40 percent, indicating more volatile pricing, even though its absolute standard deviation is smaller. Using raw standard deviation would incorrectly suggest the second zone is more stable. The coefficient of variation is the standard normalized volatility measure in statistics (Everitt, 2002) and the correct choice when comparing dispersions across populations with different means.

Everitt, B. S. (2002). The Cambridge dictionary of statistics (2nd ed.). Cambridge University Press.

Assumption 14: My Composite Scoring Weights Reflect the Strongest Predictors of Market Opportunity

Query 10, my zone opportunity matrix, uses a composite score with four components. Volume receives 30 percent weight, fare level receives 25 percent, SLA gap receives 30 percent, and wait time gap receives 15 percent. I chose these weights because SLA gap and volume together represent the core market failure signal: there is real, measurable demand and the current supply is failing to meet it. SLA gap alone being 30 percent reflects my view, supported by service quality research, that rider satisfaction is most strongly driven by wait time reliability, not price (Parasuraman et al., 1988). Fare is 25 percent rather than higher because entering a market with poor unit economics regardless of how underserved it is would not be a sound strategy. Wait time gap at 15 percent is a secondary confirmation signal that supply shortage is real, but it overlaps partially with SLA so it receives lower weight to avoid double-counting.

Parasuraman, A., Zeithaml, V. A., & Berry, L. L. (1988). SERVQUAL: A multiple-item scale for measuring consumer perceptions of service quality. Journal of Retailing, 64(1), 12-40.

Assumption 15: Negative Take Rate Trips Are Real Platform Losses, Not Data Errors

In this dataset, 18.23 percent of Uber trips and 11.61 percent of Lyft trips have a negative take rate, meaning the driver received more money than the rider paid to the platform as their base fare. My first instinct was to suspect these were data errors, but they are not. There are several real-world mechanisms that produce negative take rates on individual trips. Platforms sometimes offer driver guarantees or bonuses, particularly during low-demand periods or in underserved zones, that result in driver payments exceeding the collected base fare. Pricing promotions or discounted rides can produce the same outcome. The TLC minimum pay formula can also require a driver payment that exceeds a low base fare on a short trip. I count these trips explicitly with the negative_take_rate_pct metric and include them correctly in aggregate take rate calculations, where they appropriately reduce the overall margin.

Chen, M. K., & Sheldon, M. (2016). Dynamic pricing in a labor market: Surge pricing and flexible work on the Uber platform. Proceedings of the ACM Conference on Economics and Computation.

Assumption 16: I Apply Minimum Volume Thresholds Before Computing Volatility and Scoring

For zone-level volatility analysis in Query 9, I require at least 30 trips per zone. For the opportunity matrix in Query 10, I require at least 50 trips per zone. Below these thresholds, statistical measures like standard deviation and coefficient of variation become unreliable because a single outlier trip can dominate the calculation. A zone with three trips, one of which happened to be an airport run, would show enormous apparent volatility that tells us nothing about the structural pricing pattern in that zone. The minimum volume thresholds are conservative by design: I would rather exclude a small zone than make a confident claim about it based on insufficient data. Borough-level analyses have no threshold because boroughs always have thousands of trips and sample size is never a concern.

Field, A. (2013). Discovering statistics using IBM SPSS statistics (4th ed.). SAGE Publications.

SECTION 4

Every Calculation I Used and Why I Calculated It That Way

Every derived metric in my analysis lives in the view called v_trips_enriched. I built it this way deliberately: by centralizing all the business logic in one place, I ensure that every query uses the exact same definition of every metric. If I ever need to update a formula, I update it once in the view and every downstream query automatically picks up the change. Here I document each derived metric, its exact formula as written in SQL, and the full reasoning behind why I calculate it that way.

trip_min

```
Formula: ROUND(trip_time / 60.0, 2)
```

Converts the raw trip_time column from seconds to minutes and stores it as a clean, readable duration. The 60.0 uses decimal division to ensure floating point precision rather than integer truncation. I round to two decimal places because sub-second precision is meaningless for business interpretation but I want enough precision to avoid rounding errors in downstream aggregations.

request_to_pickup_min

```
Formula: ROUND(TIMESTAMPDIFF(SECOND, request_datetime, pickup_datetime) / 60.0, 2)
```

Calculates the wait time the rider experienced from requesting their trip to being picked up. TIMESTAMPDIFF returns the difference in seconds and I divide by 60 to convert to minutes. This is the primary service quality metric in my entire analysis. The view filter ensures this value is always between 0 and 120 minutes.

sla_5min_met

```
Formula: CASE WHEN TIMESTAMPDIFF(SECOND, request_datetime, pickup_datetime) <= 300
THEN 1 ELSE 0 END
```

A binary flag that equals 1 when the rider was picked up within five minutes of requesting, and 0 otherwise. 300 seconds equals exactly five minutes. The beauty of storing this as a 0 or 1 integer is that AVG(sla_5min_met) multiplied by 100 gives the SLA percentage for any segment without any additional CASE logic in the analysis queries. This is a standard technique for binary metric aggregation.

passenger_total_charge

```
Formula: ROUND(base_fare + tolls + bcf + sales_tax + congestion_surcharge +
airport_fee + cbd_congestion_fee + tips, 2)
```

The total amount the rider pays out of pocket for the trip. This is every dollar that leaves the rider's account, including tips. I use COALESCE to handle NULL fee columns as described in Assumption 10. This metric answers the question of what the trip actually costs the rider in full, and I use it as the denominator when computing fee burden percentages.

total_congestion_fees

```
Formula: ROUND(COALESCE(congestion_surcharge, 0) + COALESCE(cbd_congestion_fee, 0), 2)
```

Combines both government-mandated flat congestion fees into a single burden metric. I use this in Query 07 to test whether the flat fee structure creates regressive burden on short-trip or outer borough

riders. A flat fee becomes more regressive as the base fare decreases because it represents a larger percentage of a smaller total.

platform_spread

```
Formula: ROUND(base_passenger_fare - driver_pay, 2)
```

The absolute dollar margin the platform earns on each trip, before accounting for its own operating costs. A positive value means the platform kept some of the base fare after paying the driver. A negative value means the platform paid the driver more than it collected in base fare and is therefore losing money on this specific trip.

take_rate_pct (per trip only)

```
Formula: CASE WHEN base_fare > 0 THEN ROUND((base_fare - driver_pay) / base_fare *
100, 2) ELSE NULL END
```

The per-trip take rate expressed as a percentage. I compute this in the view but I use it only for per-trip analysis like decile banding in Query 04 and for counting negative take rate trips. I never use AVG(take_rate_pct) across a segment. For any segment-level take rate, I always use the aggregate SUM method.

Aggregate Take Rate (used in queries)

```
Formula: (SUM(base_passenger_fare) - SUM(driver_pay)) /
NULLIF(SUM(base_passenger_fare), 0) * 100
```

The true platform margin percentage computed over a segment, borough, or platform. NULLIF prevents a division by zero error in the unlikely event that a segment has zero total base fare. This formula appears in Queries 01, 02, 04, 08, and 11 wherever a take rate is reported at an aggregated level.

fare_per_mile

```
Formula: CASE WHEN trip_miles > 0 THEN ROUND(base_passenger_fare / trip_miles, 2) ELSE
NULL END
```

How much platform revenue was generated per mile traveled. This is a revenue intensity metric. A higher fare per mile indicates the platform is extracting more value from each unit of driver effort. I guard against division by zero with the CASE statement even though the view already filters out zero-mile trips, because defensive programming is better than relying on filter assumptions.

driver_pay_per_hour

```
Formula: CASE WHEN trip_time > 0 THEN ROUND(driver_pay / trip_time * 3600, 2) ELSE
NULL END
```

The driver's earnings rate on this specific trip, expressed as an hourly equivalent. This is computed as driver_pay divided by trip_time in seconds, multiplied by 3600 seconds per hour. It tells you how much the driver earns per hour while on a trip. It does not tell you the driver's actual take-home hourly rate, because it excludes the idle time between trips when the driver is repositioning or waiting for a request.

below_tlc_minimum

```
Formula: CASE WHEN driver_pay < ROUND((trip_miles * 0.82) + (trip_time / 60.0 * 0.57),
2) THEN 1 ELSE 0 END
```

A binary violation flag. The TLC minimum pay formula requires the driver to receive at least 0.82 dollars per mile traveled plus 0.57 dollars per minute of trip time. If driver_pay falls below this calculated minimum, the platform is in violation of the TLC minimum pay standard and this flag equals 1. Like sla_5min_met, storing it as a 0 or 1 integer means AVG(below_tlc_minimum) multiplied by 100 gives the violation rate percentage for any segment without additional logic in the queries.

is_airport_trip

```
Formula: CASE WHEN airport_fee > 0 THEN 1 ELSE 0 END
```

A binary flag derived from the presence of a positive airport fee. As explained in Assumption 5, the airport_fee column is only populated on trips to or from qualifying NYC area airports. This derived flag makes airport trip filtering simpler and more readable in downstream queries.

Coefficient of Variation

```
Formula: ROUND(STDDEV(base_passenger_fare) / NULLIF(AVG(base_passenger_fare), 0) *
100, 1)
```

The normalized volatility measure for fare prices in a given segment or zone. This is computed in Queries 09 and 11 at various levels of aggregation. The full rationale for choosing CV over raw standard deviation is documented in Assumption 13. NULLIF prevents division by zero in the extremely unlikely case that all fares in a segment are zero.

Min-Max Normalization (Opportunity Score)

```
Formula: (value - MIN(value) OVER ()) / NULLIF(MAX(value) OVER () - MIN(value) OVER
(), 0) * 100
```

Used in Query 10 to normalize each scoring dimension to a 0 to 100 scale before combining them into the composite opportunity score. This ensures that a dimension measured in dollars, such as average fare, can be combined with a dimension measured in percentages, such as SLA, without one scale dominating the other. For inverted dimensions like SLA gap, I subtract the zone's value from the maximum before normalizing, so that lower SLA produces a higher score.

SECTION 5

The Numbers I Verified and Use as My Ground Truth

I ran every one of my queries against the actual December 2025 dataset and recorded the key numbers. The table below represents the verified output I use as my reference whenever I interpret a result or write up a finding. If your run of the same queries produces numbers that differ significantly from these, the most likely causes are: the view was not created before running the query, the trip_time conversion from seconds to minutes was applied twice, or the WHERE clause filter in the view differs from the one documented here.

Total Trips	71,518	27,138
Market Share	72.49%	27.51%
Average Base Fare	$28.18	$25.17
Average Total Charge to Rider	$35.14	$31.74
Average Driver Pay per Trip	$21.60	$19.79
Average Tip	$1.26	$1.36
Average Driver Hourly Rate (on trip)	$64.71	$60.15
Aggregate Take Rate	17.95%	19.75%
Negative Take Rate Percentage	18.23%	11.61%
Average Trip Miles	4.87 miles	4.57 miles
Average Trip Duration	19.97 minutes	19.43 minutes
Average Fare per Mile	$8.48	$7.59
Average Wait Time	5.32 minutes	5.07 minutes
SLA 5-Minute Percentage	59.80%	60.55%
TLC Violation Percentage	1.05%	0.02%
Total Violation Trips	750 trips	5 trips
Average Shortfall on Violations	$5.11	N/A
December Revenue Peak Date	December 11 at $89,067	December 12 at $28,209
December Revenue Trough Date	December 26 at $39,116	December 25 at $15,634
Holiday Revenue Cliff	56.1% decline	44.6% decline
Top Entry Zone (Q10)	LaGuardia (score 100)	LaGuardia (score 100)
Vulnerable Segments (Q11)	13 segments, 45.9% of trips	0 segments

SECTION 6

The Six Most Important Things I Found

These are the findings that surprised me, challenged my initial assumptions, or changed the strategic recommendation I would make to each stakeholder. I am listing them here before the query documentation because I think it helps to know what the analysis is building toward as you read through the technical detail.

Finding 1: Uber's Margin Problem Is in the Middle of the Fare Range, Not at the Bottom

When I built Query 04 to analyze take rate by fare decile, my initial hypothesis was that Uber's negative take rates would be concentrated in the cheapest trips, the very short rides where the TLC minimum pay formula might guarantee the driver more than the rider paid. What I found was the opposite. Uber's take rate is weakest in fare deciles 4 through 6, which correspond to the 13 to 24 dollar fare range. In those deciles, between 22 and 24 percent of trips have negative take rates. Decile 1, the cheapest trips, actually performs comparably to the middle deciles. This inverts the common intuition and points to a structural pricing algorithm problem in Uber's mid-market fare band rather than a minimum pay floor problem. The additional finding that Uber's top decile, decile 10, generates 49.7 percent of all Uber margin for the month reveals how dependent Uber's economics are on a thin slice of premium trips.

Finding 2: Congestion Fee Burden Is Regressive by Trip Length, Not by Borough

I entered Query 07 expecting to find that outer borough riders, who generally pay lower base fares, would bear a disproportionate share of congestion fee burden as a percentage of their total charge. What I found was more nuanced. At the borough level, outer boroughs carry almost no congestion burden at all because their trips largely do not intersect the congestion pricing zones. Manhattan bears 84 to 87 percent of congestion fee incidence. But when I cut the data by trip distance, a very different pattern emerges. Trips under one mile pay 8 to 9 percent of their total charge in flat congestion fees. Trips over ten miles pay only about 2 percent. This means a short-trip rider inside Manhattan, who is likely to be a lower-income service worker or commuter, pays 4 to 4.5 times as much in fees relative to their total charge as someone taking a long airport trip. The policy implication is that the regressivity in the current structure comes from the flat fee design, not from geographic targeting.

Finding 3: TLC Violations Peak on Long Trips, Not Short Ones

My pre-analysis hypothesis for Query 06 was that short trips would have the highest TLC minimum pay violation rates because a high per-minute minimum on a two-minute trip seems more likely to exceed a low base fare than on a twenty-minute trip. The data proved this wrong. Under one-mile trips have a violation rate of only 0.05 percent. Trips over five miles have a violation rate of 2.14 percent, which is the highest distance band in the dataset. The mechanism is counterintuitive but logical: on a long trip in heavy New York traffic, the driver spends a large amount of time at low speed. The TLC formula charges 0.57 dollars per minute, so a one-hour congested trip generates a minimum pay requirement of around 34 dollars in time-based compensation alone, plus the mileage component. If the base fare was set before the traffic conditions materialized, it may not cover that full minimum. This finding has direct implications for where TLC enforcement resources should focus.

Finding 4: Lyft Has Zero Vulnerable Segments While Uber Has Thirteen

Query 11 synthesizes all the prior analysis into a growth quality score for each platform, borough, and time of day combination. The result is stark. Lyft has no segments that score below 40 out of 100, which is my threshold for the VULNERABLE classification. Every Lyft segment in the dataset scores 50 or higher. Uber has 13 VULNERABLE segments representing 45.9 percent of its total December trip volume. Uber's only segment scoring in the SOLID range, 60 to 79, is Manhattan Midday. The implication is that Uber's scale advantage, which is enormous at 72.5 percent market share, is built on a portfolio of segments that are structurally fragile across margin, service quality, compliance, and price stability simultaneously. A competitor does not need to attack the whole market to disrupt Uber. It only needs to target the 13 vulnerable segments.

Finding 5: Airport Riders Pay Three Times More and Get Worse Service

Query 03 revealed that airport trips generate average base fares of 115 dollars or more, which is 2.75 to 2.86 times the non-airport base fare. My first reaction was that this is justified by the longer distances involved. Then I looked at the SLA data. Airport trips in this dataset have SLA rates of 51 to 57 percent, which is 15 to 20 percentage points worse than the overall platform average. The riders paying the most are receiving the worst service. Query 05 then revealed that the airport_fee, which is the government-mandated surcharge, accounts for only 5 to 7 percent of the airport fare premium. The other 93 to 95 percent is pure platform pricing power applied to a captive demand segment. This combination of high fares, poor service, and captive demand is exactly what makes airports the top-scoring entry target in Query 10.

Finding 6: Uber Is Actively Losing Money on Staten Island Late Night

Query 11 flagged one segment as particularly alarming: Uber on Staten Island during Late Night hours. The take rate is negative 1.79 percent, which means the platform is paying drivers more per trip than it collects from riders. The SLA is 29.71 percent, meaning fewer than one in three riders is picked up within five minutes. The average wait time is 8.67 minutes. The growth quality score is 30 out of 100, placing it in the VULNERABLE tier. And this segment serves only 175 trips in an entire month. Uber is losing money, delivering terrible service, and doing so at a scale too small to ever recoup through learning curve improvements. This segment is a candidate for market exit or complete structural redesign before any capacity investment is made.

SQL Query Documentation

Each query below is documented with its business question, the stakeholder it addresses, the SQL techniques it uses, the logic behind every calculation within it, and the key findings it produced. All queries run against v_trips_enriched, which must be created first by running the view setup script. The master documentation SQL file combines all eleven queries into a single executable file.

QUERY 01

Market Position Scorecard

Business Question

What is the complete head to head competitive position of Uber and Lyft across every dimension that matters for business strategy? This is the first query I run because every subsequent finding needs to be contextualized against platform-level baselines. You cannot interpret a borough-level take rate without knowing the platform average, and you cannot evaluate a segment-level SLA without a benchmark.

Stakeholder

This query belongs to all four stakeholders. For the Uber CEO, it surfaces the margin efficiency gap versus Lyft despite Uber's volume dominance. For the Lyft CEO, it confirms the higher take rate edge. For the TLC, it shows compliance rates. For the new entrant, it establishes the competitive landscape they are entering.

Techniques and Logic

The market_share_pct calculation uses a window function, specifically SUM(COUNT(*)) OVER(), to compute the total trip count across both platforms within the same SELECT statement. This eliminates the need for a subquery and ensures the shares always sum to 100 percent regardless of which platforms are in the result set. The window function approach is both more efficient and less error-prone than the subquery alternative.

The take rate uses the aggregate SUM method as documented in Assumption 6. The negative_take_rate_pct uses SUM with a CASE expression counting trips where the per-trip take_rate_pct is negative, which is the correct use of the per-trip rate since I am counting individual trip occurrences, not averaging across them.

The below_tlc_min_pct uses AVG with a CASE expression that returns 1.0 or 0.0 rather than 1 or 0. I specify the decimal form to force floating-point arithmetic and avoid integer division, which would produce only 0 or 1 as results in some SQL environments.

Key Findings

Two rows, one for each platform, with approximately twenty KPI columns. Verified numbers: Uber 71,518 trips at 17.95 percent take rate. Lyft 27,138 trips at 19.75 percent take rate. Lyft earns more per dollar of revenue collected despite having less than four times fewer trips.

QUERY 02

Demand Heatmap by Borough and Time of Day

Business Question

Which borough and time of day combinations have the highest demand, strongest platform economics, and weakest service delivery? Where are the persistent gaps that characterize each platform's reach across the city?

Stakeholder

Lyft CEO for competitive positioning. New Entrant for identifying underserved windows. The demand heatmap is the strategic map of where each platform is winning and losing.

Techniques and Logic

Part 1 groups by borough and platform and uses RANK() OVER (PARTITION BY company_name ORDER BY COUNT(*) DESC) to rank each borough by volume within each platform independently. This means Manhattan can be rank 1 for both platforms even though they have different trip counts, because the ranking is relative to each platform's own portfolio.

Part 2 adds the time of day dimension and introduces service status labels. I defined three categories: UNDERSERVED for segments where the SLA percentage is below 50, AT RISK for 50 to 58 percent, and ADEQUATE for 58 percent and above. These thresholds were set based on my review of service quality benchmarks in the ride hailing literature (Hall et al., 2017). A 50 percent SLA means half of riders in that segment wait more than five minutes, which I consider a meaningful service failure threshold.

The ORDER BY clause at the end uses FIELD() to produce results in a logical time of day sequence rather than alphabetical order. FIELD(time_of_day, AM Rush, Midday, PM Rush, Evening, Late Night) returns the position of each value in the specified list, giving me chronological output that is much more readable than the default.

Key Findings

I found 8 UNDERSERVED segments with SLA below 50 percent, 10 AT RISK segments, and 12 ADEQUATE segments across the borough and time grid. Staten Island Late Night is the worst performer: Lyft at 19.15 percent SLA and Uber at 29.71 percent SLA. Both platforms are failing that segment simultaneously.

QUERY 03

Origin to Destination Corridor Analysis

Business Question

Which specific point-to-point corridors generate the most revenue? How do airport routes compare economically to non-airport routes? What does the borough-level revenue flow matrix look like across the five boroughs?

Stakeholder

Uber CEO for defending high-value corridors. Lyft CEO for identifying corridors where their economics are differentiated. Three separate parts address corridor analysis at progressively higher levels of geographic aggregation.

Techniques and Logic

Part 1 groups by pickup zone and dropoff zone together to identify the most valuable point-to-point pairs. I apply a HAVING COUNT(*) greater than or equal to 50 threshold because zone-level pairs with fewer trips produce statistically unreliable averages. The CONCAT function builds a readable corridor label by joining the origin and destination borough names with an arrow.

Part 2 uses the is_airport_trip flag from the view to split trips into airport and non-airport categories. This is where I quantified the airport premium paradox.

Part 3 groups by origin and destination borough rather than zone, giving a higher-level revenue flow matrix. This shows that 50.2 percent of revenue is from intra-borough trips, 35.1 percent from cross-borough trips, and 14.6 percent from outbound trips.

Key Findings

JFK to Outside NYC is the top corridor by revenue at 304 thousand dollars with a 27.32 percent take rate. Manhattan to Newark Airport has the highest take rate in the entire dataset at 34.95 percent. Bronx to Brooklyn has a take rate of only 0.42 percent, making it nearly zero-margin despite the trip volumes there.

QUERY 04

Take Rate by Fare Decile

Business Question

Where in the fare distribution is platform margin strong and where is it broken? Is the negative take rate problem concentrated in cheap short trips or spread across the fare range in a pattern that suggests a deeper pricing algorithm problem?

Stakeholder

Uber CEO primarily. This query is the diagnostic tool for understanding where Uber's 18 percent negative take rate problem actually lives in the fare distribution.

Techniques and Logic

I use NTILE(10) as a window function partitioned by company_name to assign each trip to one of ten equally sized fare buckets within each platform. Partitioning by platform is essential because Uber and Lyft have different fare distributions. If I ran a single NTILE across both platforms, Uber's higher average fare would skew which deciles contain which fare ranges, making cross-platform comparison meaningless.

The query uses a CTE chain with two stages. The first CTE called deciled assigns the decile number to each row. The second CTE called decile_summary aggregates by company and decile. This separation

keeps the logic clean and makes it easier to debug because I can test each CTE independently.

Within each decile I report both the aggregate take rate and the percentage of trips with negative take rates. These are measuring different things. The aggregate take rate tells me whether the average economics of the decile are positive or negative. The negative trip percentage tells me what fraction of individual trips are loss-making even if the aggregate is positive.

The total_margin column sums all platform_spread values within each decile. This is important because it reveals concentration: Uber's decile 10 generates 49.7 percent of total Uber margin for the month, meaning Uber's business model is extremely dependent on a small number of high-value trips.

Key Findings

Uber deciles 4 through 6, covering the 13 to 24 dollar fare range, have 22 to 24 percent negative take rate trip percentages. This is the worst band in the dataset and directly contradicts the assumption that short cheap trips are the problem. Lyft's take rate curve is smooth and healthy throughout, ranging from 16.76 percent at decile 1 to 26.32 percent at decile 10.

QUERY 05

Passenger Price Decomposition

Business Question

What is the rider actually paying for? For every dollar a rider spends on a trip, how much goes to the driver, how much to the platform, how much to government fees and taxes? Does this structure differ between platforms and between rider segments?

Stakeholder

TLC Commissioner for fee equity analysis. Lyft CEO for pricing transparency comparison. New Entrant for understanding the cost structure they need to price competitively against.

Techniques and Logic

Part 1 computes each fee component as a percentage of the total charge. I use AVG(component) divided by NULLIF(AVG(total_charge), 0) multiplied by 100 for these proportions. This gives me the average component share across all trips, which I interpret as the number of cents from each rider dollar that goes to each destination.

Part 2 computes congestion burden percentage by borough. The burden metric is AVG(total_congestion_fees) divided by AVG(passenger_total_charge) multiplied by 100. This tests the regressivity hypothesis: if outer borough trips have lower base fares but similar flat fees, the burden percentage should be higher for outer boroughs. I found instead that the borough pattern is driven by which zones intersect the congestion pricing geography, not by fare levels.

Part 3 isolates airport trips using is_airport_trip equals 1 and computes what percentage of the airport total charge is attributable to the regulated airport_fee. This quantifies exactly how much of the airport price premium is policy-mandated versus platform discretionary.

Key Findings

For every dollar Lyft collects, 62.4 cents goes to drivers, 16.9 cents to the platform, and 20.7 cents to government fees and taxes. For every Uber dollar, 61.5 cents goes to drivers, 18.7 cents to the platform. The airport_fee represents only 5 to 7 percent of the airport fare premium, confirming that the premium is almost entirely platform pricing power rather than regulatory pass-through.

QUERY 06

Driver Pay Stress Test

Business Question

Which platform, borough, and time of day combination has the highest rate of TLC minimum pay violations? How severe are the violations when they occur? And which trip characteristics are most associated with violations?

Stakeholder

NYC TLC Commissioner. The results of this query directly inform which operators and segments warrant targeted enforcement action.

Techniques and Logic

The below_tlc_minimum flag is already computed in the view, so the queries here are mostly aggregations of that flag. SUM(below_tlc_minimum) counts violations. AVG(below_tlc_minimum) multiplied by 100 gives the violation rate as a percentage. These are both simple and precise because of how I stored the flag as a binary integer.

Part 4 computes the average shortfall on violation trips using a CASE expression inside AVG: AVG(CASE WHEN below_tlc_minimum = 1 THEN minimum_due minus driver_pay END). This calculates the average underpayment amount per violation trip, which I use to estimate total underpayment across the month. Uber had 750 violations averaging a shortfall of 5.11 dollars, which totals approximately 3,832 dollars in underpaid driver earnings in December alone.

I also apply severity labels to the borough breakdown: HIGH RISK for violation rates at or above 1 percent, MODERATE for 0.5 to 1 percent, LOW for above zero but below 0.5 percent, and COMPLIANT for zero violations.

Key Findings

Uber has a violation rate of 1.05 percent compared to 0.02 percent for Lyft. Uber Brooklyn is HIGH RISK at 1.43 percent. Uber Late Night is worst by time at 2.16 percent. Violations peak on 5 plus mile trips at 2.14 percent, not on short trips. This counterintuitive result is explained by the time component of the TLC formula generating high minimum pay requirements on slow congested long trips.

QUERY 07

Congestion Burden Analysis

Business Question

How much of what NYC riders pay goes to congestion-related fees? Is the flat fee structure of the congestion surcharge and CBD congestion pricing creating disproportionate burden on any particular rider group? Where geographically are these fees being collected?

Stakeholder

NYC TLC Commissioner. This query is the equity analysis backbone of the entire project. The word congestion is literally in the project title, so this analysis needed to be both methodologically rigorous and policy-relevant.

Techniques and Logic

Part 1 computes congestion_burden_pct as AVG(total_congestion_fees) divided by AVG(passenger_total_charge) multiplied by 100. I also compute congestion_vs_base_pct as the ratio to base fare rather than total charge, which gives a different perspective on how much of the platform-controlled price is effectively offset by regulatory fees. I label each borough with an equity status based on the burden percentage threshold: HIGH BURDEN above 10 percent, MODERATE BURDEN between 5 and 10 percent, LOW BURDEN below 5 percent.

Part 2 cuts the data by trip distance band using a CASE expression with six categories from under one mile to ten or more miles. This is the key test of the regressivity hypothesis. If the flat fees are regressive on short trips, I expect to see congestion_burden_pct declining as the distance band increases. That is exactly what I found.

Part 3 filters to zones with a positive average CBD fee and ranks them by total CBD fees collected. This shows the geographic footprint of the congestion pricing program in terms of revenue concentration by zone.

Key Findings

Trips under one mile pay 8 to 9 percent of their total charge in flat congestion fees. Trips over ten miles pay only 2 percent. The regressivity is trip-length based. Manhattan bears 84 to 87 percent of congestion fee incidence but this is geographic alignment, not inequitable targeting. The CBD fee is concentrated on 95.2 percent of Manhattan trips and is nearly absent from outer borough trips.

QUERY 08

Holiday Demand Stress and Revenue Volatility

Business Question

How severely does December holiday demand volatility affect platform revenue? When exactly is the peak, when is the trough, how deep is the cliff, and how does the recovery pattern differ between platforms?

Stakeholder

Uber CEO for understanding cliff exposure. Both platforms for operational planning and driver supply management around holiday periods.

Techniques and Logic

This query uses the most advanced window function logic in the entire project. The first window function is LAG(daily_base_revenue) OVER (PARTITION BY company_name ORDER BY trip_date), which returns the previous day's revenue for the same platform. This enables me to compute a day over day percentage change without joining the table to itself. The first day of the month returns NULL for LAG because there is no prior day, which is expected and handled with a CASE expression in the final SELECT.

The second window function computes a 7-day rolling average using AVG(daily_base_revenue) OVER with a frame of ROWS BETWEEN 6 PRECEDING AND CURRENT ROW. This smooths out day to day noise and reveals the underlying demand trend. Each row's 7-day average includes the current day plus the six preceding days for that platform.

The third window function computes a running cumulative revenue total using SUM(daily_base_revenue) OVER with ROWS UNBOUNDED PRECEDING. This gives a month to date accumulation that is useful for tracking when each platform crosses revenue milestones.

The RANK() window function ranks each day by revenue within each platform, with rank 1 being the peak day. I use RANK rather than ROW_NUMBER because RANK correctly handles ties, assigning the same rank to days with identical revenue.

Key Findings

Uber peaks on December 11 at 89,067 dollars in base revenue and troughs on December 26 at 39,116 dollars, a cliff of 56.1 percent. Lyft peaks on December 12 at 28,209 dollars and troughs on December 25 at 15,634 dollars, a cliff of 44.6 percent. The different trough dates suggest different rider base behavior: Uber's riders may be more concentrated in business travel that resumes post-Christmas, while Lyft's riders appear to cancel on Christmas Day itself.

QUERY 09

Price Volatility Analysis

Business Question

Which zones and boroughs have the widest fare variance? Are high-volatility zones also zones where riders have few alternative transportation options? When during the day does pricing become most unpredictable?

Stakeholder

TLC Commissioner for surge pricing audit. New Entrant for identifying zones where pricing is stable enough to model a launch economics case.

Techniques and Logic

I compute the coefficient of variation as STDDEV(base_passenger_fare) divided by NULLIF(AVG(base_passenger_fare), 0) multiplied by 100. As documented in Assumption 13, this normalization is essential for comparing volatility across zones with different average price levels. The

HAVING COUNT(*) greater than or equal to 30 threshold ensures that the STDDEV calculation is based on enough observations to be reliable.

I classify each zone into volatility categories based on CV thresholds: STABLE below 35 percent, MODERATE VOLATILITY between 35 and 60 percent, and HIGH VOLATILITY above 60 percent. I also assign an entry signal to each zone based on the combination of volatility and SLA: VOLATILE PLUS UNDERSERVED for high-CV zones with SLA below 55 percent, and ENTRY OPPORTUNITY for stable zones with decent volume and weak SLA.

Part 3 filters specifically to Late Night trips for a focused volatility analysis of the most stress-prone window. Late night is the period when driver supply is tightest, demand is least predictable, and surge pricing is most likely to occur (Cohen et al., 2016).

Key Findings

East Tremont in the Bronx has the highest zone-level CV at 168.8 percent. The Garment District has the highest single fare recorded at 660 dollars. Staten Island late night has a 38.6 percent fare surge premium with only 19.15 percent SLA for Lyft, the worst combination in the dataset. Queens borough has the only CV above 100 at the borough level, at 103.5 percent for Uber.

QUERY 10

Zone Opportunity Matrix

Business Question

If a new platform were entering the NYC ride hailing market, which specific zones offer the best combination of high demand, high fares, and weak incumbent service? Where are wheelchair accessible vehicle needs going unmet? What does a phased entry strategy look like based on the data?

Stakeholder

New Market Entrant exclusively. This is the culminating strategic query for that perspective.

Techniques and Logic

I use a four-CTE chain to build the composite opportunity score. The first CTE called zone_metrics aggregates all raw metrics per zone including volume, fare levels, SLA, wait times, violation rates, and WAV request and match counts. The second CTE called normalized applies min-max normalization to each dimension using window functions. The third CTE called scored computes the weighted composite. The final SELECT adds ranking and labeling.

The min-max normalization uses (value minus MIN(value) OVER ()) divided by NULLIF(MAX minus MIN, 0) times 100. For dimensions where a lower value is better for entry opportunity, specifically SLA and wait time, I invert the normalization by subtracting the zone value from the maximum before dividing. This means a zone with SLA 30 percent scores near 100 on the SLA gap dimension while a zone with SLA 70 percent scores near 0, correctly reflecting that low incumbent SLA is a greater entry opportunity.

The WAV analysis in Part 2 computes wav_request_rate_pct as the percentage of all trips where a rider requested a wheelchair accessible vehicle. I include an interpretation label that distinguishes between suppressed demand, where requests are near zero likely because riders stopped trying, and fulfillment failure, where requests exist but are not matched to WAV vehicles. This distinction matters for policy and strategy differently.

Key Findings

LaGuardia Airport scores 100 out of 100 on the opportunity matrix, driven by 1,848 trips, a 62.78 dollar average base fare, a 41-minute average wait time, and a weak incumbent SLA. JFK Airport scores 78.6 as the second-ranked zone. My recommended entry strategy is: Tier 1 target both airports with reliability-focused operations, Tier 2 build outer borough capacity in the MONITOR zones, and Tier 3 differentiate with a WAV fleet in the Bronx and Manhattan where unmet WAV demand is measurable.

QUERY 11

Growth Quality Score

Business Question

Which platform, borough, and time of day segments have genuinely sustainable growth across all four dimensions simultaneously? Which segments are fragile in ways that invite competitive disruption? This is the synthesis query that ties together every other analysis into a single strategic verdict per segment.

Stakeholder

Uber CEO for internal portfolio health review. New Entrant for identifying which specific Uber segments are fragile enough to be worth targeting.

Techniques and Logic

The query uses a three-CTE chain. The first CTE called segment_base aggregates all raw metrics by platform, borough, and time of day. The second CTE called scored applies the scoring logic to assign 0 to 25 points per dimension. The third CTE called final_scored adds two RANK window functions: one overall and one within platform using PARTITION BY company_name.

The scoring logic uses nested CASE expressions with both a take rate threshold and a negative take rate percentage threshold as joint conditions for the margin dimension. The reason for this joint condition is that a segment could have a high average take rate driven by a few very profitable trips while still having 30 percent of trips losing money. Both dimensions must be healthy simultaneously to earn the full score.

The compliance dimension uses violation_pct thresholds at 0, 0.25, 0.5, 1.0, and 2.0 percent. I set these thresholds conservatively because even a one-percent violation rate represents a meaningful regulatory risk at scale: Uber with 71,000 trips and a one-percent violation rate means 710 drivers were underpaid in a month.

Key Findings

Lyft has no vulnerable segments and 63.4 percent of its trips in SOLID segments scoring 60 or above. Uber has 13 VULNERABLE segments representing 45.9 percent of its trip volume. The absolute worst segment in the dataset is Uber Brooklyn Late Night at a growth quality score of 25 out of 100 due to a 20.76 percent negative take rate, a 2.61 percent TLC violation rate, and a 53.26 percent SLA. The single Uber SOLID segment is Manhattan Midday at 65 points.

SECTION A

Output Section Labels and APA References

Each query produces result sets labeled with a section column so that you can identify which query generated which output when running the master file.

(platform-level rows)	Query 01	Market Position Scorecard
Borough Summary	Query 02, Part 1	Borough by Platform Summary
Borough x Time	Query 02, Part 2	Borough by Time by Platform Heatmap
Top Corridors	Query 03, Part 1	Top 20 Zone-Level Origin-Destination Pairs
Airport Segment	Query 03, Part 2	Airport versus Non-Airport Economics
Borough Matrix	Query 03, Part 3	Borough-to-Borough Revenue Flow Matrix
Decile Summary	Query 04	Take Rate by Fare Decile
Platform Decomposition	Query 05, Part 1	Full Fee Breakdown by Platform
Borough Fee Burden	Query 05, Part 2	Congestion Fee Burden by Borough
Airport Fee Structure	Query 05, Part 3	Airport versus Non-Airport Fee Structure
Platform Summary	Query 06, Part 1	TLC Compliance Platform Summary
Borough Breakdown	Query 06, Part 2	Violation Rate by Borough
Time of Day Breakdown	Query 06, Part 3	Violation Rate by Time of Day
Short Trip Analysis	Query 06, Part 4	Violation Rate by Distance Band
Borough Congestion Burden	Query 07, Part 1	Congestion Fee Burden by Borough
Distance Band Burden	Query 07, Part 2	Burden Percentage by Trip Distance
CBD Zone Footprint	Query 07, Part 3	CBD Fee by Zone, Top 20
Daily Revenue Trend	Query 08, Part 1	Daily Revenue with LAG and Rolling Average
Peak vs Trough	Query 08, Part 2	Holiday Cliff Summary
Zone Volatility	Query 09, Part 1	Zone-Level Coefficient of Variation, Top 40
Borough Volatility	Query 09, Part 2	Borough-Level Coefficient of Variation Summary
Late Night Volatility	Query 09, Part 3	Late Night Volatility by Borough
Opportunity Matrix	Query 10, Part 1	Top 30 Zone Entry Opportunities
WAV Gap Analysis	Query 10, Part 2	Wheelchair Access Gap by Borough
Growth Quality Score	Query 11, Part 1	Full Segment Quality Ranking
Platform Quality Summary	Query 11, Part 2	Platform Head-to-Head Quality Comparison

APA References

Black Car Fund. (2024). About the Black Car Fund. Black Car Fund. https://www.blackcarfund.org

Chen, M. K., & Sheldon, M. (2016). Dynamic pricing in a labor market: Surge pricing and flexible work on the Uber platform. Proceedings of the ACM Conference on Economics and Computation. https://doi.org/10.1145/2940716.2940798

Cohen, P., Hahn, R., Hall, J., Levitt, S., & Metcalfe, R. (2016). Using big data to estimate consumer surplus: The case of Uber. National Bureau of Economic Research Working Paper No. 22627. https://doi.org/10.3386/w22627

Everitt, B. S. (2002). The Cambridge dictionary of statistics (2nd ed.). Cambridge University Press.

Field, A. (2013). Discovering statistics using IBM SPSS statistics (4th ed.). SAGE Publications.

Hall, J. V., Horton, J. J., & Knoepfle, D. T. (2017). Pricing in designed markets: The case for ride-sharing. Working Paper, Harvard University.

Metropolitan Transportation Authority. (2025). Central business district tolling program: Frequently asked questions. MTA. https://new.mta.info/project/central-business-district-tolling-program

Mishel, L. (2018). Uber and the labor market: Uber drivers' compensation, wages, and the scale of Uber and the gig economy. Economic Policy Institute. https://www.epi.org

Murphy, T. (2019). Data type conventions in relational databases. ACM SIGMOD Record, 48(2), 12-19.

New York City Taxi and Limousine Commission. (2023). 2023 TLC factbook. NYC TLC. https://www.nyc.gov/site/tlc

New York City Taxi and Limousine Commission. (2024). High volume for-hire vehicle rules: Chapter 59B of the rules of the city of New York. NYC TLC. https://www.nyc.gov/site/tlc/businesses/high-volume-for-hire-services.page

New York City Taxi and Limousine Commission. (2024). Minimum payment standard for high volume for-hire vehicle drivers. NYC TLC. https://www.nyc.gov/site/tlc/for-drivers/minimum-pay-rate.page

New York State Department of Taxation and Finance. (2019). Tax Law Section 1299-B: Congestion surcharge on certain for-hire vehicles. New York State. https://www.tax.ny.gov

Parasuraman, A., Zeithaml, V. A., & Berry, L. L. (1988). SERVQUAL: A multiple-item scale for measuring consumer perceptions of service quality. Journal of Retailing, 64(1), 12-40.

Rahm, E., & Do, H. H. (2000). Data cleaning: Problems and current approaches. IEEE Data Engineering Bulletin, 23(4), 3-13.

Schaller, B. (2017). Empty seats, full streets: Fixing Manhattan's traffic problem. Schaller Consulting.

Varian, H. R. (2014). Big data: New tricks for econometrics. Journal of Economic Perspectives, 28(2), 3-28. https://doi.org/10.1257/jep.28.2.3